

Reproducible RPC Inference Across Resource-Constrained ARM Single-Board Computers

Hudson Moreira

Affiliation and contact details to be confirmed by the authors

Abstract—Splitting a quantized language model across small local computers is attractive when one device has insufficient memory, but networked execution can exchange computation for communication overhead. This paper reports a reproducible, deliberately narrow evaluation of the RPC backend of `llama.cpp` on homogeneous ARM single-board computers. We compare five valid executions of a notebook-only baseline, five of a single Labrador board, and five of a notebook coordinating two Labrador RPC workers. The selected workload uses Qwen2.5-1.5B-Instruct Q5_K_M, a fixed short prompt, deterministic sampling, and a frozen runtime. The logs verify that both RPC workers received assigned layers and executed graph operations for each selected request. Under this configuration, two-worker RPC is slower than the isolated Labrador baseline: 168.67 s versus 43.86 s of process time. The result is a bounded observation, not a claim about all models, networks, or distributed systems. We release the raw evidence, reconstruction scripts, build recipe, and failure inventory for review.

Index Terms—edge AI, consumer electronics, ARM SBC, local inference, RPC, reproducibility

I. INTRODUCTION

Large language model inference is increasingly relevant to local and consumer devices, where privacy, connectivity, and resource ownership can motivate execution near the user. Quantization reduces model storage and memory requirements, while model partitioning can combine several constrained devices. However, a partitioned request must move intermediate state and coordinate remote computation; therefore, adding devices need not reduce latency.

This work studies that trade-off in a controlled laboratory setup. It does not propose a new partitioning algorithm or claim a general scalability limit. Instead, it asks a narrower question: for one quantized instruction model, one short deterministic prompt, and a homogeneous ARM testbed, does coordinating two RPC workers complete the request faster than running the same model on one Labrador board?

The contributions are: (1) an auditable dataset with explicit inclusion and exclusion rules; (2) direct evidence that both selected RPC workers participated in the same inference, rather than merely accepting a connection; and (3) a reproducible comparison separating prompt processing, generation, actual token count, and process time. The answer is negative for the tested configuration: two-worker RPC is slower than the isolated board.

II. RELATED WORK

The RPC backend used here is an experimental feature of `llama.cpp` that exposes remote devices and allows a client to offload computation to one or more servers [1]. Qwen2.5 provides the model family context for the selected instruction-tuned checkpoint [2]. Prior edge-inference work addresses broader system designs: EdgeShard formulates device selection and model partitioning for heterogeneous edge resources [3], while MDI-LLM studies model-distributed inference and recurrent pipeline parallelism across low-power devices [4]. Those studies motivate collaborative edge inference but do not replace a measurement of this specific RPC implementation, hardware, and workload. Our contribution is consequently an evaluation artifact and bounded baseline rather than a new distributed-inference method.

III. TESTBED AND PROTOCOL

A. Hardware and software

The testbed contains homogeneous Labrador ARM single-board computers connected by a switched 100 Mb/s Ethernet segment. The article uses pseudonymous roles: the notebook is the client; Labrador-A and Labrador-B are the two RPC workers. The internal raw artifacts retain the original operational identifiers, while this paper avoids publishing IP addresses and hostnames. The notebook executes the client, scheduler, local CPU backend, and any model portion not assigned remotely; it is not a passive controller.

The model is Qwen2.5-1.5B-Instruct in Q5_K_M GGUF format (SHA-256 `b4666107...896f8c`). The fixed client binary has SHA-256 `13b3c2e9...348d69`. The ARM package was cross-compiled from `llama.cpp` commit `68e79bd8`, with RPC enabled, native CPU detection disabled, and OpenMP, CUDA, Vulkan, Metal, and RDMA disabled. The local compilation variant applies the preserved `-O0` patch to the common and RPC HTTP compilation targets. The package uses GCC 12 and a Debian 12 arm64 sysroot; the ABI audit reports maximum GLIBC 2.34 and GLIBCXX 3.4.30.

B. Frozen workload

Every selected case used context 512, four threads, `-ngl 99`, a maximum of 32 output tokens, temperature zero, seed 42, `--ignore-eos`, and no warmup. The prompt was

TABLE I
SELECTED VALID EXECUTIONS AND PROCESS-TIME STATISTICS. VALUES ARE RECONSTRUCTED FROM RAW LOGS; SD IS SAMPLE STANDARD DEVIATION.

Condition	n	Mean (s)	SD (s)	Min (s)	Max (s)
Notebook	5	5.69	0.04	5.61	5.71
Labrador	5	43.86	0.43	43.52	44.58
RPC-2	5	168.67	0.16	168.49	168.83

“Write exactly one short sentence about distributed computing.” The runtime reports 38 prompt tokens and, for every included case, 32 actually generated tokens. Each process was newly started; disk caches were not cleared, so the experiment is not a cold-disk benchmark. The process-time metric includes startup, model loading, RPC setup, prompt processing, generation, and residual work not separately instrumented.

C. Validity and selection

The five notebook cases come from matrix-validation-1; the five isolated ARM cases come from matrix-validation-5; and the five RPC cases combine two valid cases from matrix-validation-7 with three valid cases from rpc2-complete-1. These groups share model hash, client hash, runtime source commit, prompt, sampling parameters, and context. They do not form one consecutive campaign: the launcher and health-check procedure changed before the final three RPC cases, so campaign provenance is retained and the groups are not presented as uninterrupted repetitions.

IV. WORKER PARTICIPATION EVIDENCE

The RPC selection requires more than an open TCP port or a successful client exit. In each included RPC log, the client assigns layers 0–14 to Labrador-A and layers 15–27 to Labrador-B. The paired server-log slices contain graph computation and recomputation events, tensor transfers, and buffer allocation for both hosts. Each request records 38 evaluated prompt tokens, 32 predicted tokens, and normal completion with exit code zero. The earlier instrumented pilot independently established the same participation pattern; it is cited as validation evidence and was not repeated for this paper.

The selected cases have 426 relevant operation records in the Labrador-A server slice and 435 in Labrador-B under the reconstruction script’s graph/tensor event rule. These counts are evidence of execution activity, not a claim that the two servers performed identical work. The notebook remains part of the computation, so the comparison is notebook-only versus one ARM board versus notebook-plus-two-workers, not one board versus an entirely remote cluster.

V. RESULTS

Table II separates the runtime’s prompt and generation measurements. The reported generation rate is the runtime’s predicted-token count divided by its generation time; it uses 32 generated tokens, not the process-time denominator. We do not

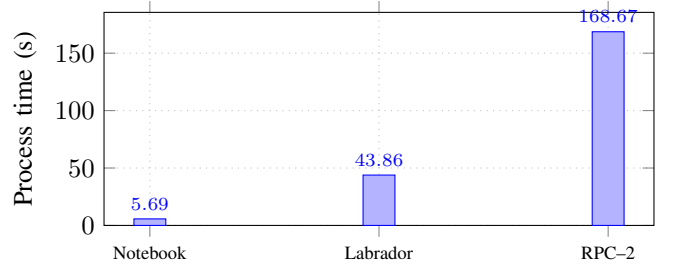


Fig. 1. Mean process time for the five selected valid cases per condition. Error bars are reported in Table I; the plot emphasizes scale, not statistical significance.

TABLE II
MEAN PHASE METRICS FOR VALID SELECTED CASES.

Condition	Prompt (s)	Generation (s)	Tokens	Runtime gen. (tok/s)
Notebook	1.222	2.426	32	13.19
Labrador	15.948	19.377	32	1.65
RPC-2	31.076	36.914	32	0.87

call the remaining process time “loading time”: loading and first-token timestamps were not independently instrumented.

The five selected RPC cases all completed, giving a conditional success rate of 100% for the selected set. This is not an operational reliability estimate: earlier attempts include connection failures, loading timeouts, terminal-input interruptions, and a process abort after partial generation. The full CSV preserves those denominators and classifications. The paper therefore reports performance conditioned on valid completion separately from the broader operational failure history.

VI. DISCUSSION AND LIMITATIONS

The observed result is specific and reproducible: in this configuration, RPC-2 is slower than the isolated Labrador board. The latency difference is consistent with a combination of remote graph execution, intermediate-state communication, serialization, and coordination. The dataset does not isolate those mechanisms; in particular, the study does not justify attributing the result exclusively to the 100 Mb/s link.

The experiment has one model, one short prompt, one quantization, one network segment, and five valid cases per condition. Data were collected in distinct campaigns, and detailed logging was used for RPC validation. Loading, time to first token, peak memory, and a direct communication-only cost are absent. Memory snapshots are before/after observations, not peak measurements. The study does not evaluate user-facing services, quality, energy, privacy leakage, concurrent requests, or model sizes that require more memory. Local services on consumer devices motivate the work, but no particular consumer application was tested.

VII. CONCLUSION

We presented a reproducible audit of a small ARM RPC inference setup. The raw logs establish that both RPC workers participated in each of five selected two-worker requests.

Under the frozen Qwen2.5-1.5B Q5_K_M workload, RPC-2 required 168.67 s of process time on average, compared with 43.86 s for a single Labrador board. This is an observation about the tested configuration, not a universal statement about distributed inference. The released evidence and failure inventory provide a basis for future work on phase-level loading and communication measurements without overstating scalability.

ACKNOWLEDGMENT AND AI-USE DISCLOSURE

The authors should confirm all author metadata, affiliations, and funding before submission. Generative-AI assistance was used to help organize repository evidence, draft portions of this manuscript, and prepare analysis code; the authors remain responsible for verification, technical claims, citations, and final text. This disclosure is included for author review in accordance with IEEE guidance on AI-generated text [5].

REFERENCES

- [1] ggml-org, “llama.cpp rpc backend,” GitHub repository documentation, 2026, accessed 2026-10-04. [Online]. Available: <https://github.com/ggml-org/llama.cpp/tree/master/tools/rpc>
- [2] Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2025. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [3] M. Zhang, J. Cao, X. Shen, and Z. Cui, “Edgeshard: Efficient llm inference via collaborative edge computing,” *arXiv preprint arXiv:2405.14371*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.14371>
- [4] D. Macario, H. Seferoglu, and E. Koyuncu, “Model-distributed inference for large language models at the edge,” *arXiv preprint arXiv:2505.18164*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.18164>
- [5] IEEE, “Guidelines for artificial intelligence (ai)-generated text,” 2024, accessed 2026-10-04. [Online]. Available: <https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/>